

Canonical ML Workflow Framework Research

Overview

A reusable, algorithm-agnostic ML platform should be built on a canonical workflow that separates universal ML lifecycle stages from data-type-specific, algorithm-specific, and deployment-specific variations.[cite:446][cite:449] Across established practice, the recurring backbone is iterative rather than strictly linear: business understanding, data acquisition and understanding, modeling, deployment, monitoring, and acceptance or feedback loops.[cite:449][cite:451] Framework design should therefore treat the pipeline as a dependency graph with controlled branching points, not as a single fixed sequence of notebook steps.[cite:446][cite:449]

The most durable abstractions come from three sources: classical project frameworks such as CRISP-DM and TDSP, production MLOps patterns that operationalize models with pipelines and monitoring, and model-persistence practices that preserve training artifacts for later scoring and retraining.[cite:446][cite:449][cite:374] This document derives a first-principles ML workflow from that foundation and maps it to reusable framework modules, plugins, and agentic decision points.[cite:446][cite:449][cite:374]

Canonical lifecycle

End-to-end stages

The canonical ML lifecycle can be represented as the following ordered but iterative stages.[cite:446][cite:449][cite:451]

1. Problem framing and success criteria.[cite:446][cite:449]
2. Dataset acquisition and access setup.[cite:446][cite:449]
3. Data understanding and schema inspection.[cite:446][cite:449]
4. Exploratory data analysis.[cite:446][cite:451]
5. Data validation and quality assessment.[cite:446][cite:451]
6. Data cleaning and correction.[cite:446][cite:451]
7. Preprocessing and transformation.[cite:446][cite:451]
8. Feature engineering.[cite:446][cite:451]
9. Feature selection or dimensionality reduction.[cite:446][cite:451]
10. Dataset splitting and evaluation design.[cite:446][cite:451]
11. Model family selection.[cite:446][cite:451]
12. Hyperparameter tuning.[cite:446][cite:451]
13. Training.[cite:446][cite:451]
14. Validation and error analysis.[cite:446][cite:451]
15. Evaluation and model comparison.[cite:446][cite:451]
16. Explainability and robustness checks.[cite:451]
17. Packaging and persistence of artifacts.[cite:374][cite:445]
18. Deployment and operationalization.[cite:449][cite:374]

19. Monitoring of data, predictions, and system health.[cite:449][cite:374]

20. Retraining, rollback, and lifecycle governance.[cite:449][cite:374]

Stage reference table

Stage	Purpose	Inputs	Outputs	Mandatory	Typical tools/libraries	Depends on
Problem framing	Define objective, target, constraints, KPIs	Business question, domain context	ML problem statement, success metrics	Yes [cite:446] [cite:449]	Docs, tickets, notebooks	None
Dataset acquisition	Obtain raw data and access rights	Source systems, files, APIs	Raw datasets, access metadata	Yes [cite:446] [cite:449]	SQL, pandas, Spark, storage connectors	Problem framing
Data understanding	Inspect schema and semantics	Raw dataset	Data dictionary, schema profile	Yes [cite:446] [cite:449]	pandas, profiling tools	Acquisition
EDA	Understand distributions and patterns	Profiled data	Plots, hypotheses, risks	Usually yes [cite:446] [cite:451]	pandas, seaborn, matplotlib	Understanding
Data validation	Assess quality and consistency	Raw/profiled data	Validation report, issue list	Yes [cite:446] [cite:451]	custom checks, Great Expectations-style tools	Understanding
Data cleaning	Correct or remove bad records	Validated data	Clean dataset	Usually yes [cite:446] [cite:451]	pandas, numpy, SQL	Validation
Preprocessing	Transform values into model-ready form	Clean dataset	Encoded/scaled/transformed data	Yes [cite:446] [cite:451]	scikit-learn preprocessing	Cleaning
Feature engineering	Create predictive representations	Clean or transformed data	Feature matrix	Often yes [cite:446] [cite:451]	pandas, feature stores, domain code	Preprocessing
Feature selection	Reduce noise or dimension	Feature matrix	Reduced feature set	Optional [cite:446] [cite:451]	scikit-learn, PCA, model-based selectors	Feature engineering
Split design	Prevent leakage and estimate generalization	Dataset with labels/context	Train/val/test or CV design	Yes [cite:446] [cite:451]	scikit-learn splitters	Features/problem type
Model selection	Choose candidate algorithms	Problem definition, feature set	Candidate models	Yes [cite:446] [cite:451]	scikit-learn, XGBoost, PyTorch	Split design
Hyperparameter tuning	Optimize model settings	Candidate models, train/val plan	Tuned configs	Optional but common [cite:446] [cite:451]	GridSearchCV, Optuna	Model selection

Stage	Purpose	Inputs	Outputs	Mandatory	Typical tools/libraries	Depends on
Training	Fit models to training data	Train set, params	Fitted models	Yes [cite:446] [cite:451]	scikit-learn, TensorFlow, PyTorch	Selection/tuning
Validation	Tune thresholds and diagnose errors	Fitted model, validation data	Thresholds, error analysis	Yes [cite:446] [cite:451]	metrics libs, notebooks	Training
Evaluation	Measure final performance	Final model, test data	Metrics report	Yes [cite:446] [cite:451]	scikit-learn metrics	Validation
Model comparison	Pick champion model	Metrics across candidates	Selected model	Usually yes [cite:446] [cite:451]	experiment trackers, tables	Evaluation
Explainability	Understand model behavior and trust	Fitted model, data	Feature importance, explanations	Optional but often required [cite:451]	SHAP, PDP, coefficients	Evaluation
Packaging/persistence	Save model and metadata	Selected model, code, environment info	Serialized artifacts and configs	Yes for deployment [cite:374] [cite:445]	joblib, pickle, skops, ONNX	Model selection complete
Deployment	Operationalize model with pipeline	Artifacts, pipeline code	Batch or realtime service	Optional for research, mandatory for production [cite:449] [cite:374]	Docker, APIs, schedulers, MLflow	Packaging
Monitoring	Track health, drift, failures	Live data, predictions, logs	Dashboards, alerts	Mandatory in production [cite:449] [cite:374]	dashboards, logs, metrics stacks	Deployment
Retraining/governance	Refresh or replace model safely	Monitoring signals, new data	New model versions, rollback plan	Optional in prototype, mandatory in production lifecycle [cite:449] [cite:374]	CI/CD, registries, orchestration	Monitoring

Fine-grained atomic ML operations

Problem framing operations

- Define business objective.[cite:446][cite:449]
- Define prediction target or anomaly concept.[cite:446][cite:449]
- Define unit of analysis, such as row, window, user, asset, or document.[cite:446][cite:449]
- Define latency requirement: batch, near-real-time, or realtime.[cite:449]
- Define acceptable error trade-offs and KPIs.[cite:446][cite:449]
- Define deployment environment and constraints.[cite:449][cite:374]

Dataset acquisition operations

- Identify source systems.[cite:446][cite:449]
- Create extract query or connector.[cite:446][cite:449]
- Snapshot or version raw data.[cite:374][cite:445]
- Capture source metadata and lineage.[cite:449]
- Validate access permissions and privacy scope.[cite:449]

Data understanding operations

- Infer schema.[cite:446]
- Classify columns by semantic type: identifier, numeric, categorical, timestamp, text, image path, graph edge, sensor tag.[cite:446][cite:451]
- Compute row and column counts.[cite:446]
- Inspect sample records.[cite:446]
- Build preliminary data dictionary.[cite:446][cite:451]

EDA operations

- Univariate summaries.[cite:446][cite:451]
- Distribution plots and skew checks.[cite:451]
- Correlation and association checks.[cite:451]
- Segment analysis by class, mode, geography, or asset.[cite:451]
- Temporal trend inspection for time data.[cite:451]
- Hypothesis generation for features, leakage, and quality problems.[cite:451]

Data validation operations

- Null and missingness profile.[cite:446][cite:451]
- Duplicate row and key collision checks.[cite:446][cite:451]
- Range checks.[cite:446][cite:451]
- Domain rule checks.[cite:446][cite:451]
- Timestamp continuity or ordering checks.[cite:451]
- Referential integrity checks across tables.[cite:451]
- Label consistency checks.[cite:451]
- Leakage risk checks.[cite:451]

Data cleaning operations

- Duplicate removal.[cite:446][cite:451]
- Missing value treatment: drop, impute, carry-forward, model-based imputation, sentinel value.[cite:446][cite:451]
- Outlier handling: cap, winsorize, flag, preserve, or remove.[cite:446][cite:451]
- Datatype correction.[cite:446]
- Timestamp normalization and timezone alignment.[cite:451]

- Unit conversion and unit harmonization.[cite:451]
- Categorical value normalization.[cite:451]
- Text normalization or OCR cleanup where relevant.[cite:451]

Preprocessing operations

- Train/inference schema alignment.[cite:446][cite:451]
- Categorical encoding: one-hot, ordinal, target, hashing.[cite:451]
- Scaling: standardization, min-max, robust scaling, normalization.[cite:451]
- Log or power transforms.[cite:451]
- Tokenization or embedding preparation for text.[cite:451]
- Resizing or augmentation preparation for images.[cite:451]
- Resampling or windowing for time series.[cite:451]
- Sequence padding or truncation.[cite:451]

Feature engineering operations

- Aggregate features.[cite:451]
- Ratio and interaction features.[cite:451]
- Lag features for temporal data.[cite:451]
- Rolling statistics.[cite:451]
- Trend, slope, volatility, and baseline deviation features.[cite:451]
- Spatial features, graph features, or embeddings.[cite:451]
- Domain-rule features and context IDs.[cite:451]

Feature selection operations

- Variance filtering.[cite:451]
- Correlation pruning.[cite:451]
- Mutual information or univariate tests.[cite:451]
- Embedded importance selection.[cite:451]
- Recursive feature elimination.[cite:451]
- Dimensionality reduction such as PCA or autoencoder bottlenecks.[cite:451]

Split and validation operations

- Random train/validation/test split.[cite:446][cite:451]
- Stratified split for class preservation.[cite:451]
- Group-aware split.[cite:451]
- Chronological split for time series.[cite:451]
- Cross-validation or rolling-origin validation.[cite:451]
- Holdout definition for final acceptance.[cite:449][cite:451]

Modeling operations

- Candidate algorithm shortlist.[cite:446][cite:451]
- Baseline model training.[cite:446][cite:451]
- Hyperparameter search space definition.[cite:446][cite:451]
- Model fitting.[cite:446][cite:451]
- Threshold calibration when scores must become labels.[cite:451]
- Probability calibration where needed.[cite:451]

Evaluation operations

- Metric computation.[cite:446][cite:451]
- Error slicing and subgroup analysis.[cite:451]
- Confusion matrix or residual analysis.[cite:451]
- Ranking of candidates.[cite:451]
- Robustness, sensitivity, and stress tests.[cite:451]
- Explainability artifacts.[cite:451]

Packaging and deployment operations

- Serialize model artifact.[cite:374][cite:445]
- Save preprocessing artifact and feature list.[cite:374][cite:445]
- Save code version, dependency versions, and training metadata.[cite:445][cite:374]
- Wrap scoring interface as function, batch job, or API.[cite:449][cite:374]
- Containerize runtime if needed.[cite:374][cite:445]
- Register model and deploy.[cite:449]

Monitoring and retraining operations

- Monitor service health and latency.[cite:449]
- Monitor input schema and data drift.[cite:449]
- Monitor prediction distribution and threshold behavior.[cite:449]
- Monitor business metrics or downstream acceptance.[cite:449]
- Trigger retraining, rollback, or investigation.[cite:449][cite:374]

Branching by problem type

Shared backbone across problem types

Most ML problem types still share problem framing, acquisition, understanding, validation, cleaning, preprocessing, feature creation, split design, training, evaluation, packaging, deployment, monitoring, and retraining.[cite:446][cite:449][cite:451] The branching occurs mainly in target handling, validation strategy, feature design, model families, and metrics.[cite:446][cite:451]

Problem-type comparison

Problem type	Common stages	Distinct preprocessing/feature needs	Validation strategy	Typical metrics
Regression	Full standard pipeline [cite:446][cite:451]	Continuous target checks, possible target transform [cite:451]	Random/group/time split depending on data [cite:451]	MAE, RMSE, R-squared [cite:451]
Classification	Full standard pipeline [cite:446][cite:451]	Label encoding, imbalance treatment, thresholding [cite:451]	Stratified CV often useful [cite:451]	Accuracy, F1, ROC-AUC, PR-AUC [cite:451]
Anomaly detection	Similar pipeline but often no labels or sparse labels [cite:451]	Normal-only training, contamination estimation, score thresholding [cite:451]	Time-aware or score-calibration validation [cite:451]	Precision@k, recall on known events, alert rate [cite:451]
Forecasting / time series	Same backbone with stronger temporal discipline [cite:451]	Lag, rolling windows, seasonality, exogenous variables [cite:451]	Chronological split, rolling-origin validation [cite:451]	MAE, RMSE, MAPE, sMAPE, pinball loss [cite:451]
Clustering	Similar early stages but no target [cite:451]	Distance scaling, embedding choice, clusterability checks [cite:451]	Internal validation or downstream utility [cite:451]	Silhouette, Davies-Bouldin, business usefulness [cite:451]
Recommendation	Shared backbone plus interaction modeling [cite:451]	User-item matrix, implicit/explicit feedback features [cite:451]	Temporal/user holdout [cite:451]	Precision@k, Recall@k, NDCG, MAP [cite:451]
Deep learning (image)	Shared lifecycle with heavier data and training stack [cite:451]	Resize, normalize, augment, class balancing [cite:451]	Stratified or grouped splits [cite:451]	Accuracy, F1, mAP, top-k accuracy [cite:451]
NLP	Shared lifecycle with language-specific preprocessing [cite:451]	Tokenization, embeddings, truncation, prompt/template design [cite:451]	Stratified/grouped/task-specific validation [cite:451]	Accuracy, F1, BLEU, ROUGE, perplexity depending on task [cite:451]
Reinforcement learning	High-level exception to standard supervised flow [cite:451]	Environment definition, reward design, simulation [cite:451]	Episodic evaluation, offline/online tests [cite:451]	Reward, regret, stability, safety [cite:451]

Branching notes

Regression and classification stay closest to the textbook tabular pipeline.[cite:446][cite:451] Anomaly detection often changes the assumptions at split, training, validation, and threshold-selection stages because labels may be sparse and training may intentionally use only healthy or normal data.[cite:451] Forecasting keeps most lifecycle stages but replaces random split logic with chronological evaluation and adds window-based feature engineering, seasonality analysis, and horizon-specific scoring.[cite:451]

Branching by data type

Shared preprocessing across data types

All data types still require schema understanding, validation, versioning, split design, packaging, deployment, and monitoring.[cite:446][cite:449] The major changes happen in representation, cleaning rules, and feature engineering.[cite:451]

Data-type comparison

Data type	Shared preprocessing	Type-specific preprocessing	Type-specific feature engineering
Tabular	Null handling, encoding, scaling [cite:451]	Category harmonization [cite:451]	Ratios, interactions, bins [cite:451]
Time-series	Validation, scaling, splitting [cite:451]	Resampling, alignment, gap handling, chronology checks [cite:451]	Lags, rolling stats, trends, seasonality [cite:451]
Industrial historian data	Same as time-series plus context validation [cite:451]	Operating mode filtering, tag alignment, sensor sanity rules [cite:451]	Drift, stability, process relationships, context-aware windows [cite:451]
Sensor telemetry	Same as time-series [cite:451]	Sampling consistency, noise filtering [cite:451]	Frequency features, vibration or signal stats [cite:451]
Images	Label checks, splits [cite:451]	Resize, normalize, augment [cite:451]	CNN features, embeddings, patches [cite:451]
Audio	Label checks, splits [cite:451]	Resampling, denoising, silence trimming [cite:451]	Spectrograms, MFCCs, embeddings [cite:451]
Text	Basic validation [cite:451]	Tokenization, normalization, stopword or subword handling [cite:451]	n-grams, embeddings, contextual vectors [cite:451]
Graphs	ID and relation validation [cite:451]	Edge cleaning, graph construction [cite:451]	Degree, centrality, node embeddings [cite:451]
Event logs	Timestamp and case validation [cite:451]	Sessionization, sequence extraction [cite:451]	Path frequencies, dwell time, transition features [cite:451]

Historian and industrial data notes

Industrial historian data behaves like time series but usually adds operating modes, asset hierarchy, process constraints, alarm context, and domain rules that materially change preprocessing and anomaly logic. [cite:451] As a result, historian pipelines often require context filters, temporal windows, and rule-assisted exclusion of known fault periods before model training.[cite:451]

Decision points

The pipeline contains recurring decision nodes that determine branching behavior.[cite:446][cite:451] These are strong candidates for framework policy objects and, in some cases, agentic reasoning support.[cite:446] [cite:449]

Core decision questions

Decision point	Required information	Typical outcomes
Is the problem supervised, unsupervised, semi-supervised, or reinforcement-based?	Business objective, label availability [cite:446][cite:451]	Choose target-dependent vs target-free pipeline
Is there a target column?	Schema, business definition [cite:446] [cite:451]	Regression/classification path vs clustering/anomaly path
Is data time-ordered?	Timestamp availability and semantics [cite:451]	Chronological split and temporal features
Is the primary data type tabular, text, image, audio, graph, or event log?	Schema and storage format [cite:451]	Load corresponding preprocessing plugin
Are labels trustworthy and complete?	Label audit, coverage review [cite:451]	Standard training vs weak supervision vs anomaly strategy

Decision point	Required information	Typical outcomes
Is imbalance severe?	Class distribution [cite:451]	Reweighting, resampling, threshold changes
Should missing values be removed or imputed?	Missingness pattern, sample size, domain meaning [cite:451]	Drop, simple impute, forward-fill, model-based impute
Should outliers be removed, capped, flagged, or preserved?	Domain context, objective, rarity significance [cite:451]	Remove noise vs preserve signal
Is scaling required?	Algorithm family and feature units [cite:451]	Apply scaler or skip
Is feature selection needed?	Dimensionality, multicollinearity, compute budget [cite:451]	Select/reduce features or keep all
Is threshold calibration required?	Score-producing model, alert budget [cite:451]	Score-to-label calibration step
What deployment mode is required?	Latency, volume, integration target [cite:449]	Batch job, API, stream service
When should retraining occur?	Drift, decay, new data cadence, governance policy [cite:449][cite:374]	Scheduled or event-triggered retraining

Algorithm-specific variations

Shared versus specific behavior

Algorithm selection changes only part of the workflow, not the whole lifecycle.[cite:446][cite:451] The reusable framework should therefore preserve the same stage skeleton while swapping algorithm-specific policies and plugins for training, preprocessing, thresholds, and metrics.[cite:446][cite:451]

Regression

Regression usually adds target sanity checks, optional target transformation, regression-specific metrics, residual diagnostics, and feature-importance analysis tuned to continuous outcomes.[cite:451] Scaling may matter for linear, kernel, and neural models but less for tree ensembles.[cite:451]

Classification

Classification typically adds label encoding, imbalance treatment, probability or threshold calibration, classwise metrics, and confusion-matrix analysis.[cite:451] Validation strategy often favors stratification to preserve class proportions.[cite:451]

Anomaly detection

Anomaly detection often changes the pipeline more deeply than standard supervised tasks.[cite:451] It may require normal-only training, contamination estimation, threshold selection, score ranking, event aggregation, and specialized evaluation that reflects alert volume and known-event recall rather than ordinary accuracy.[cite:451]

Forecasting

Forecasting requires lag features, rolling windows, horizon definitions, exogenous feature alignment, seasonality analysis, temporal split discipline, and metrics aligned to forecast error over future horizons.[cite:451] Random shuffling is usually invalid because it leaks future information into training.[cite:451]

Dependency graph

High-level dependency structure

The lifecycle is best represented as a graph with iteration loops.[cite:446][cite:449]

- Problem framing -> acquisition.[cite:446][cite:449]
- Acquisition -> understanding -> validation -> cleaning.[cite:446][cite:451]
- Cleaning -> preprocessing -> feature engineering.[cite:446][cite:451]
- Feature engineering -> split design -> model selection/tuning/training.[cite:446][cite:451]
- Training -> validation -> evaluation -> comparison.[cite:446][cite:451]
- Selected model -> explainability -> packaging -> deployment.[cite:374][cite:449]
- Deployment -> monitoring -> retraining or rollback.[cite:449][cite:374]
- Monitoring or evaluation failures can loop back to acquisition, cleaning, feature engineering, or model selection depending on root cause.[cite:446][cite:449]

Practical dependency rules

- Packaging cannot happen before the preprocessing recipe and selected model are frozen.[cite:374][cite:445]
- Deployment should include the data pipeline, not just the model object.[cite:449]
- Monitoring must observe both system health and model/data behavior, because production issues can come from infrastructure as well as model drift.[cite:449][cite:374]
- Retraining should preserve lineage: training data snapshot, code, environment, and validation evidence.[cite:445][cite:374]

Framework abstraction

Core framework modules

The core framework should contain operations that appear in nearly every ML workflow.[cite:446][cite:449][cite:451]

- Project/problem specification.
- Data ingestion and versioning.
- Schema profiling and validation.
- Cleaning primitives.
- Generic preprocessing primitives.
- Split strategies.
- Experiment tracking and comparison.
- Artifact persistence and metadata capture.[cite:374][cite:445]
- Deployment adapters.[cite:449]
- Monitoring hooks and retraining triggers.[cite:449][cite:374]

Plugin layers

The plugin system should handle operations that vary by algorithm, data type, or deployment target.
[cite:446][cite:449][cite:451]

Plugin category	Examples
Data-type plugins	Time-series resampling, text tokenization, image augmentation, graph construction
Problem-type plugins	Forecast horizon builder, anomaly thresholding, recommendation candidate generation
Algorithm plugins	Tree model trainer, linear model trainer, isolation forest trainer, transformer trainer
Evaluation plugins	Forecast metrics, ranking metrics, anomaly event metrics
Deployment plugins	Batch scorer, REST API, streaming scorer, edge export
Monitoring plugins	Drift checks, alert-budget checks, latency monitors, business KPI monitors

Recommended architectural separation

The most reusable decomposition is: core deterministic pipeline engine, data-type extension pack, problem-type extension pack, algorithm pack, and deployment pack.[cite:446][cite:449] This avoids duplicating the lifecycle while still allowing strong specialization at the exact stages where specialization is needed.
[cite:446][cite:451]

Mapping to an agentic framework

Where agents help

An agentic system should be introduced only where interpretation, choice, or synthesis is genuinely needed.
[cite:449][cite:451] Deterministic operations remain preferable for repeatability, speed, and auditability.
[cite:449][cite:374]

Operation classification

Stage	Deterministic operations	Reasoning-heavy operations	Suitable for LLMs	Better as pure Python
Problem framing	Template generation	Map business goal to ML task	Yes, for requirement synthesis	No for final ground-truth spec
Data understanding	Schema stats, profiling	Infer semantic meaning of columns	Yes, with human review	Profiling itself
Validation	Range/null/duplicate checks	Decide whether anomaly is data issue or domain truth	Yes, as assistant	Checks themselves
Cleaning	Imputation/normalization code	Choose treatment policy	Yes, policy recommendation	Actual transformation
Feature engineering	Standard feature transforms	Propose domain features	Yes, especially for domain ideation	Actual feature computation
Model selection	Running benchmarks	Choosing candidate families under constraints	Yes, assisted shortlist	Training/evaluation loops
Evaluation	Metric computation	Error interpretation and trade-offs	Yes	Metric calculation
Packaging/deployment	Serialization, container builds	Choose persistence and serving mode	Yes, for design guidance	Artifact creation

Stage	Deterministic operations	Reasoning-heavy operations	Suitable for LLMs	Better as pure Python
Monitoring	Drift/stat collection	Determine root-cause hypotheses and retraining priority	Yes	Metric/log computation
Retraining	Workflow execution	Decide when to retrain and what to promote	Yes, policy support	CI/CD execution

Recommended agent roles

- **Problem Framing Agent:** converts business questions into task definitions, targets, constraints, and KPIs.
- **Data Understanding Agent:** interprets schema, column semantics, and likely preprocessing needs.
- **Validation Reasoning Agent:** explains data-quality failures and recommends treatment policies.
- **Feature Ideation Agent:** proposes candidate features based on domain and data type.
- **Model Strategy Agent:** recommends candidate algorithms, validation strategy, and tuning scope.
- **Evaluation Agent:** summarizes trade-offs, slices failure modes, and supports champion/challenger selection.
- **MLOps Agent:** advises on persistence, deployment target, monitoring, and retraining policy.[cite:449][cite:374]

These agents should not replace deterministic code paths. They should configure, justify, or prioritize them. [cite:449][cite:374]

Recommendations for a reusable algorithm-agnostic platform

Design principles

- Build around a canonical lifecycle graph, not a single notebook sequence.[cite:446][cite:449]
- Separate mandatory stages from optional optimizations.[cite:446][cite:451]
- Treat data type, problem type, and algorithm family as orthogonal branching dimensions.[cite:451]
- Persist not only models but also preprocessing logic, feature lists, environment metadata, and validation evidence.[cite:445][cite:374]
- Make deployment include the full pipeline needed for inference, not just the model object.[cite:449][cite:374]
- Keep agentic reasoning above deterministic operators rather than inside every low-level transformation.[cite:449]

Minimal core modules

A practical first implementation of the framework should include:

1. Project specification module.
2. Data ingestion and snapshot module.
3. Schema and validation module.
4. Cleaning and preprocessing module.
5. Feature pipeline module.
6. Split and validation-strategy module.

7. Training and tuning module.
8. Evaluation and comparison module.
9. Artifact persistence module.[cite:374][cite:445]
10. Deployment module.[cite:449]
11. Monitoring and retraining module.[cite:449][cite:374]

Highest-value plugin packs

The highest-value plugins for broad reuse are usually:

- Time-series and industrial data plugins.
- NLP plugins.
- Image/deep-learning plugins.
- Anomaly-detection plugins.
- Forecasting plugins.
- Deployment-target plugins for batch, API, and streaming use.[cite:449][cite:451]

Final synthesis

The canonical ML workflow accepted across mainstream data-science practice is an iterative lifecycle that starts with problem framing and raw data acquisition, moves through understanding, validation, cleaning, preprocessing, feature creation, split design, modeling, evaluation, and explainability, and then continues into packaging, deployment, monitoring, and retraining.[cite:446][cite:449][cite:451] This lifecycle is stable enough to serve as the backbone of a reusable ML framework, while branching behavior should be encapsulated in plugins driven by data type, learning paradigm, algorithm family, and deployment target.[cite:446][cite:449]

The strongest abstraction is therefore a two-layer architecture: a deterministic core lifecycle engine plus plugin packs and policy/agent layers for branching decisions.[cite:446][cite:449][cite:374] That design preserves scientific rigor and production readiness while creating a clear place for selective agentic orchestration where reasoning adds value.[cite:449][cite:374]